CS30401 Network Lab 1: Improving TCP for LEO Satellite Networks

Logistics

- All submissions are due at **23:59 (KST) on October 13, 2026**.
- One member per team submits on behalf of the team, as a single zip file named `Lab1_{TeamID}.zip` on KLMS.
- In addition, every team member submits the individual Contribution Survey (see Submissions).
- All submitted materials must be written in **English**.
- Discussion and Q&A: post on Classum with the “Lab, Q&A” tag.
- Two documents come with this lab: this handout, which describes the task and the rules, and the Lab Environment Guide, which explains how to install and use the simulation environment.

Goal of the Lab

This lab focuses on understanding and improving TCP performance in Low Earth Orbit (LEO) satellite networks. Your team will carry out a small-scale research on your own; that is, you will experience finding one or more problems, propose ideas to address them, and evaluate your solutions. You will write a report documenting your findings and the results of your experiments.

LEO satellite networks, such as Starlink, OneWeb, and Amazon Kuiper, now serve millions of users, and most of LEO traffic runs over TCP, which was designed for terrestrial wired networks. An end-to-end path through a LEO constellation, however, can be quite different from a terrestrial one; for example, the LEO satellites move and thus the path between two points on Earth may change continuously. In this lab, you will investigate whether the state-of-the-art TCP implementations work well in the LEO settings, and what should change if it does not.

As we cannot experiment with real LEO satellite networks directly, unfortunately, we instead provide a simulation environment of a LEO satellite network. Your team will run TCP in it, identify what TCP performance problems arise, explain why, and propose and evaluate a fix.

Through this lab, you will:

- Experience a small-scale research project as a team, going from an open question to a concrete, evaluated result.
- Understand how TCP actually behaves, and how its mechanisms respond to a network environment it was not designed for.

- Practice making claims backed by evidence: reproducing a problem in simulation, explaining its mechanism, and evaluating your own ideas, including where it does not work.
- Practice making your work reproducible, so that someone else can regenerate your results from your code.

Background: LEO Satellite Networks

LEO satellites orbit a few hundred to about two thousand kilometers above the Earth. At 550 km (that is, the altitude of Starlink's first shell), a satellite circles the Earth in about 95 minutes at roughly 7.6 km/s (or 27,360 km/h). Each satellite sees only a small area of the ground, and a given location sees a given satellite for only a few minutes. A LEO network therefore needs hundreds to thousands of satellites, arranged in *shells* of orbital planes, to cover the Earth continuously [1], [3].

A LEO network uses three types of radio links:

- **Ground-satellite links (GSL)** between a ground terminal and a satellite currently in view. The link to a user terminal is often called the *user link*.
- **Feeder links** between a satellite and a *gateway*, a ground station connected to the terrestrial Internet.
- **Inter-satellite links (ISL)**, laser links between adjacent satellites.

Traffic can be carried in two ways (Figure 1). In a bent-pipe design, a satellite immediately relays the user's traffic down to a gateway near the user, and the rest of the path is the terrestrial Internet. With ISLs, traffic can stay in space: it goes up to a satellite, travels across several ISLs, and comes down near its destination. Because light travels about 1.5 times faster in vacuum than in optical fiber, an ISL path can be potentially faster than terrestrial fiber over long distances [2].

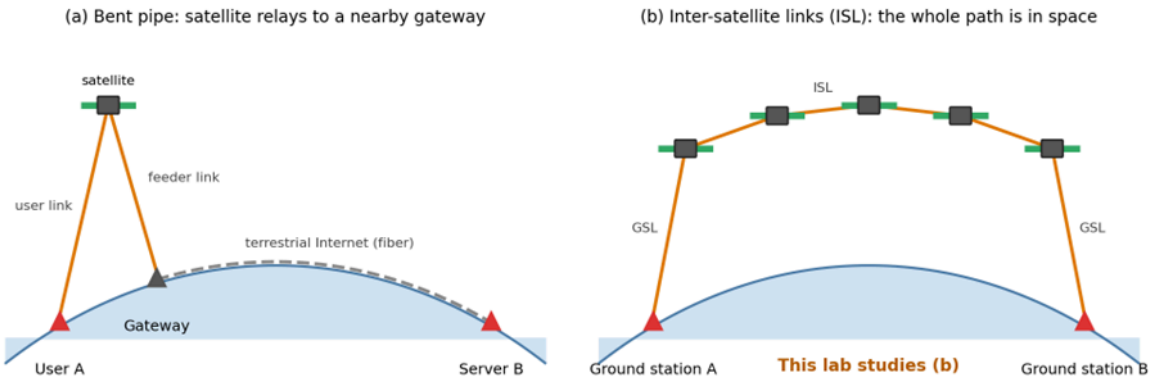


Figure 1. Two ways to carry traffic over a LEO network. This lab studies (b): two ground stations connected only through satellites.

In this lab, we consider only the ISL path (Figure 1(b)). Two ground stations are connected only through satellites: traffic goes up from one ground station to a satellite in view, travels across ISLs, and comes down at the other ground station. There are no gateways and no terrestrial segments on the path.

What makes this path different from a terrestrial one is that everything moves. The satellite a ground station uses changes, the ISL route between the two ground stations changes, and the length of the path changes continuously. Radio links the satellites use may change easily due to weather conditions or other factors. How TCP reacts to such a path is what you will investigate in this lab.

The Lab Environment

All teams use the simulation environment we provide: **ns-3.48** with the LEO satellite modules of **Hypatia** [1] (ported to ns-3.48 by the TA) together with tools to generate constellations and run experiments. The ns-3 tool is the third generation of the ns network simulator family, which has long been used in both academic and industrial research for simulating network protocols and environments. Hypatia is a recent academic framework that provides the specific models and tools needed for LEO satellite network simulation. It is distributed as a prebuilt Docker image, `ghcr.io/yeonjong21/cs30401-lab1:2026f-v1`, and can also be installed manually. The **Lab Environment Guide** (which is distributed along with this main lab handout) explains installation, usage, and starting points.

The environment starts from the following defaults:

Item	Default	Basis
Constellation	Starlink shell 1: 72 orbital planes × 22 satellites = 1,584 satellites, altitude 550 km, inclination 53°, minimum elevation angle 25°	Hypatia [1], from SpaceX's FCC filing
ISLs	+Grid: each satellite links to two neighbors in its plane and one in each adjacent plane	Hypatia [1], [3]
Ground stations	Seoul and London (any number and location can be set)	Example
Routing	Shortest path by distance over ISLs only, recomputed every 100 ms	Hypatia [1]
Links	Every ISL has the same data rate and drop-tail queue size, and so does every GSL. `make_run.sh` gives both kinds the same values; edit the run configuration to set them separately.	Chosen by you, with a stated basis

You may change the ground stations, the number of satellites, link rates, queue sizes, TCP variants, traffic, and any part of ns-3 or the satellite modules. Every value you choose must have a stated basis (see STEP 1).

Lab Instructions

Your team will carry out the following three steps in order, and report the outcome of each.

STEP 1: Problem Statement

Reproduce, by simulation, one or more concrete problems that arise when TCP runs over the LEO path described above, and explain them.

Demonstrate the problem. The problem can be broadly any technical problem of TCP-related performance that arises when TCP runs over a LEO satellite network. It may come from the motion of the satellites, from conditions on the links such as weather, from many users sharing the network, or from anything else that makes a LEO path different from a terrestrial one. It may also show up in many forms: lower throughput, higher or more variable delay, unfair sharing between users, or anything else that matters to applications, to name a few.

Your described problem(s) must be directly related to the specific designs of one or more of the existing state-of-the-art TCP implementations. The choice of TCP variant(s) is up to you, but you must justify why you think the study of the chosen variant(s) is relevant and timely. Also, your problem statement must be clear and precise. A vague claim, such as "TCP does not work well over LEO networks", is not considered a proper problem statement. Show which TCP mechanism misbehaves, under what conditions, and with what consequence: what degrades, by how much, and why. Go beyond reporting the symptom; explain the mechanism.

You may extend the environment. The provided environment models only a limited set of aspects of a LEO network (see Section 1.2 of the Lab Environment Guide); for example, it has no link interruptions, no changes in link capacity, and no random losses. If your problem depends on behaviors outside of what is modeled, you may add it. What you add is held to the same standard as any other parameter: how long, how often, and how strong the behavior is must have a stated basis. You must also show that the problem you observe comes from the behavior itself and not from the way you implemented it.

STEP 2: Idea Proposal

Propose one or more ideas that address the problem(s) you identified in STEP 1. For each idea, your proposal must make the following clear:

- **What problem(s) your idea(s) target.** Tie the mechanism of your idea(s) back to the specific problem(s) you identified in STEP 1.
- **What it assumes.** State what must hold for your idea to work, and discuss whether it is realistic. In particular, your idea may rely on information that TCP does not normally have, such as satellite positions or the times at which routes will change, to name a few. If so, explain who needs this information (the sender, the receiver, or the satellite network), how they would obtain it in a real deployment (for example, by computing it from published orbits, or by being informed by the network operator), and how accurate it needs to be.
- **Where it would be deployed.** Sender-side TCP, receiver-side TCP, both, or inside the satellite network as its operator? Each choice carries a different deployment cost; discuss it.

STEP 3: Experiment and Evaluation

Evaluate how your proposed idea(s) actually address the problem(s).

Platform. Use the provided environment (see the Lab Environment Guide). Report the version of the environment you used: the Docker image tag you used (`2026f-v1`) or, for a manual installation, the ns-3 version and the commit of the environment repository. Changes to ns-3 or to the satellite modules are submitted as a patch file.

Your evaluation should include:

- **Metrics.** State which metrics you use and why they reflect the problem you are addressing.
- **Quantitative comparison.** Show with concrete numbers how much your idea(s) improve the problem(s) compared to the baseline, the same scenario without your idea(s).
- **Conditions and costs.** Show under which conditions your idea(s) help and under which it does not, and what it costs (for example, delay, fairness, or complexity).
- **Limitations.** Discuss the limitations of your idea(s) (e.g., additional delay, increased complexity), including scenarios where it may not perform well or may introduce new challenges.

Submissions

Your submission has two parts: a **team submission** and an **individual Contribution Survey** from every member.

The team submission is a single zip file named `Lab1_{TeamID}.zip` with the following structure:

```
Lab1_{TeamID}.zip
├── report.pdf
├── README.md
└── code/
```

1. Report

One report covering all three steps, with the following structure:

1. Introduction
2. Problem Statement
3. Proposed Idea(s)
4. Experiment Setup: your simulation environment and the basis for each parameter you used
5. Evaluation: metrics, results, and quantitative comparison against the baseline
6. Conclusion
7. References
8. Appendix (optional): material that does not fit in the main body, such as parameter derivations or additional results

Format.

- Use the **IEEE conference template** (two-column, 10-pt Times) without changing its font size, margins, or spacing. Both the LaTeX and Word versions are accepted; Overleaf also provides the LaTeX version. Submit the result as `report.pdf`.
- Put your Team ID and the names of all members under the title.
- The main body (Sections 1 to 6) is limited to **6 pages**. References and appendices do not count toward the limit. This is a limit, not a target. Content beyond the page limit will not be graded.
- Every figure must be readable in print: label both axes and give units.

2. Code (code/)

All files needed to reproduce every result in your report:

- Your scenario programs, configuration files (`config_ns3.properties`, `schedule.csv`) or the scripts that generate them, and your implementation.
- If you generated your own constellation state, the exact command that generated it. Do not include the state itself or simulation logs.
- If you modified ns-3 or the satellite modules, a patch file made with `make_patch.sh` (see the Lab Environment Guide), rather than the whole tree.
- Analysis and plotting scripts.
- If your analysis or plotting scripts depend on external Python packages, manage them with uv and include `pyproject.toml` and `uv.lock` in `code/`, so that a TA can install everything with `uv sync` and run your scripts with `uv run`. If your scripts use only the Python standard library, no dependency file is needed.
- Do **not** include the ns-3 source tree or build artifacts.

3. README.md

Your `code/` and `README.md` alone must be sufficient to reproduce every result in your report. The README should contain:

- How to apply your patch (if any) and build.
- The exact commands to run your experiments, and roughly how long they take.
- A mapping from each figure and table in your report to the command that produces it.

We recommend providing a single script (e.g., `run_all.sh`) that regenerates all of your results. One good way to check your submission is to reproduce it yourself before you submit: unzip `Lab1_{TeamID}.zip` into a new folder, start a fresh environment, and follow only your `README.md`, using nothing that is not in the zip. If you get the same results as in your report, the TAs will too.

How we check reproducibility. After the deadline, a TA starts the course Docker image (`2026f-v1`) with a fresh ns-3 volume, applies your patch if there is one, follows your README word for word, and compares the regenerated figures and tables with your report.

If the results cannot be reproduced, reproducibility receives no credit, and any figure or table that cannot be regenerated is treated as unsupported when we grade your evaluation.

4. Contribution Survey (individual)

Each team member submits the Contribution Survey individually via Google Forms. The link is available on KLMS and in this document.

- Due: 23:59 (KST) on October 13, 2026.
- It asks briefly what you did and what each of your teammates did.
- Your responses are read only by the TA and are never shared with your teammates.
- Submission is mandatory. When contributions within a team differ significantly, we may conduct further investigation (e.g., interviews with team members separately).
- link: https://docs.google.com/forms/d/e/1FAIpQLSfb047CX0F-8sB6ych7c0HoaxBBWAmj9T_ZisNA0BjwMCdHdA/viewform

Schedule

- The team submission is due at **23:59 (KST) on October 13, 2026**.
- Late submission policy:
  - Up to 24 hours late: 80% credit. Up to 48 hours late: 50% credit. After that, no credit.
- The individual Contribution Survey is due at **23:59 (KST) on October 13, 2026**.
- All teams present their work on **October 15, 2026**. Details will be announced later.
  - Each team has at most 10 minutes.
  - Every team member must present part of the talk, ideally equally sharing the presentation time.

Grading Criteria

Quality of the problem statement(s). Whether the problem(s) are clearly defined and shown to actually occur in the LEO environment.

Quality of the proposed idea(s). Whether the idea(s) target the cause you identified, whether its assumptions are stated and reasonable, and whether you are clear about what was already known and what you added.

Soundness of the simulation environment and experiment. Whether your simulation environment is adequate for the problem you study, whether each parameter has a stated basis, and whether your evaluation supports the claims you make.

Reproducibility. Whether a TA can regenerate your results from your `code/` and `README.md` alone, following the procedure described in Submissions.

Clarity of the report. Whether the problem, the idea(s), and the results are clearly stated, and whether the report follows the required format and page limit.

Presentation. Whether your work is delivered effectively within the time limit, and whether every member contributes.

Academic Integrity

Plagiarism. Do not copy ideas, text, figures, or code from others' work, the Web, or other teams. Each team's problem statement, idea, environment, and evaluation must be its own. If you reuse code from the provided environment or from the ns-3 examples, say so in your README.

Citation. Cite all related work you rely on, including the simulation environment (Hypatia [1]). Each citation of a paper must include:

- Authors, title, conference or journal name, and year
- Example: Kassing, Simon, et al. "Exploring the 'Internet from space' with Hypatia." *Proceedings of the ACM Internet Measurement Conference*, 2020.

Use of AI tools. You are free to use AI tools for any part of this lab, but you are responsible for what you submit. AI tools sometimes hallucinate references that do not exist, and they often fail to translate your intent into code correctly. Your report, code, and references must tell a consistent story: the numbers you report must be the ones your code produces, and the papers you cite must exist and say what you claim they say. "The AI wrote it" is not an explanation for a result you cannot account for. We will follow the strict academic standards regarding the results of AI-generated content; that is, *strict rejection* of any work that contains hallucinated data/information in the critical arguments/arguments of the work. Thus, please take extra care to verify all AI-generated content before including it in your submission.

Frequently Asked Questions

Q. Are we allowed to use AI tools such as ChatGPT or Claude?

Yes, you may use them freely, but you are responsible for what you submit. See Academic Integrity for details.

Q. Do we have to use Docker?

No, but we recommend it. You may install the environment manually (see the Lab Environment Guide). Docker is there to save you from setup problems: the image contains the whole environment, already built and tested, and runs the same way on Windows, macOS, and Linux. If you run into installation or configuration problems on your computer, use Docker instead. Either way, state the environment you used in your `README.md`.

Q. How detailed and realistic does our model have to be?

That depends on your problem. We do not require any particular level of realism. What matters is not complexity but sufficiency: model only what your problem statement and your

idea(s) actually require. The environment models some aspects of a LEO network and not others (see the Lab Environment Guide). If your problem needs something it does not model, add it and justify its parameters.

Q. Can we change the constellation?

Yes. You can change the number and locations of the ground stations and the number of satellites with the provided options, and other orbital parameters by editing the state generator. Justify every change.

Q. Our idea does not bring a dramatic improvement. Can we still get a good grade?

How practical and effective your idea is does count, but it does not determine everything. Analyzing why your idea yields only a marginal improvement (which part of the mechanism limits it, and under what conditions) has research value of its own. A clear analysis of a modest result is worth more than an unsupported claim of a large one.

Q. Our results differ every time we run. What should we do?

The provided scenario is deterministic: the same inputs give identical results. If you add randomness (random loss, random start times, and so on), control it with the seed (`simulation_seed` in `config_ns3.properties`, or ns-3's `RngSeedManager` and `--RngRun`), and report results over several seeds.

Q. Any tips for working as a team?

Using GitHub is not required in this lab, but we ***highly recommend*** that your team work together in a private GitHub repository. If you are new to the shell or Git, MIT's *The Missing Semester of Your CS Education* (https://missing.csail.mit.edu/) is a good place to start.

Q. A team member is not contributing.

The individual Contribution Survey asks every member about their own and their teammates' contributions, and we read every response. If a serious problem arises during the lab, message a TA and we will do our best to handle it.

Q. Where do we ask questions?

Post on Classum with the "Lab, Q&A" tag. Please read the Guidelines on How to Ask Questions for Lab before posting.

References

1. S. Kassing, D. Bhattacherjee, A. B. Aguas, J. E. Saethre, and A. Singla, "Exploring the 'Internet from space' with Hypatia," *Proceedings of the ACM Internet Measurement Conference (IMC)*, 2020.
2. M. Handley, "Delay is Not an Option: Low Latency Routing in Space," *Proceedings of the ACM Workshop on Hot Topics in Networks (HotNets)*, 2018.
3. D. Bhattacherjee and A. Singla, "Network Topology Design at 27,000 km/hour," *Proceedings of the ACM Conference on Emerging Networking Experiments and Technologies (CoNEXT)*, 2019.